This function returns a struct

```
% Author: Jiale Jiang
% Editor: Jiale Jiang

function data = SpeedKalmanFilter(t,v)
    data = struct();
    % intitialize those vectors so their size match;
    t = t(:);
    v = v(:);
    % this is the true state-measurement transformation matrix;
    Hk = [1 0];
    % this is the covariance matrix of the measruement noise since
    % measurement only measures speed, so we only have 1 element;
    R = 1;
    % this is the covariance matrix of the process noise;
    Q = ones(2,2);
    % the true state of our system, contains the car's speed and
    % acceleration;
    x_true = zeros(2,1);
    % this is the predicted state of our system, before the measurement
    % innovation;
    x_before = [0; 0];
    % this is the predicted state of our system, after the measurement
    % innovation;
    x_after = zeros(2,1);
    % this is the covariance matrix of our next predicted state;
    P_after = zeros(2,2);
    % this q is initialized as the small unprediceted acceleration change
    % as we assume constant acceleration during a small time interval;
    q = 1;

    % start from index 2 since differential based calculation
    for k = 2:length(t)
        % Z is the measured state matrix;
        Z = v(k);
        % dt is the time intervals;
        dt(k) = t(k) - t(k-1);
        % this is the classic white-jerk model for process noise;
        % detailed derivation here :
        % https://www.mathworks.com/help/fusion/ref/initvisionbboxkf.html
        % or you can see my hand written note attatched when submission;
        Q = q .* [dt(k).^3./3 dt(k).^2./2;
                  dt(k).^2./2 dt(k)];
        % this is the state-transition matrix, transform last true state
        % into current true state thru acceleration discrete integration;
        F = [1 dt(k);
             0 1];
        % apply state transition
        x_before = F * x_after;

        % sensor measurement(input speed vector is our
```

```
        % "measurement" here), recursive formula;
        Y = Z - Hk * x_before;

        % calculate covariance matrix before the measurement innovation;
        P_before = F * P_after * F.' + Q;

        % calculate the optimal kalman gains
        S = Hk * P_before * Hk.' + R;
        K = P_before * Hk.'/S;
        % perform innovation from measurement;
        x_after = x_before + K * Y;
        % calculate covariance matrix after measurement innovation;
        P_after = ...
            (eye(2) - K * Hk) * P_before * (eye(2)- K*Hk).'...
            + K * R * K.';
        % save the predicted acceleration and velocity;
        data.predicted_accel(k) = x_after(2);
        data.filtered_speed(k) = x_after(1);
        % update new process noise covariance matrix coefficient using the
        % estimated variance of all acceleration changes;
        q = var(gradient(data.predicted_accel,t(1:k)).*dt(k))./dt(k);
    end
end
```

Published with MATLAB® R2024b